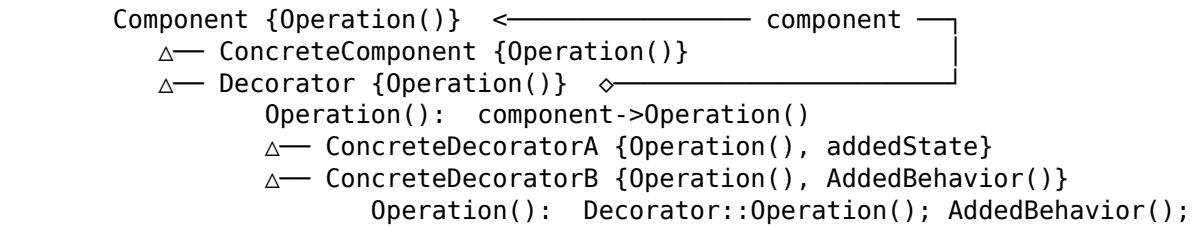
Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Decorator kalıbı: amaç ve motivasyon (ekrandan, 00:10:00)
  - Amaç: nesnelere çalışma anında (dinamik olarak) ek sorumluluk atamak. Kalıtımla eklenen özellikler derleme anında belirlenir ve aynı alt sınıftan oluşturulan bütün nesnelere aynı biçimde geçer; Decorator ise tek tek nesnelerin farklı biçimde özelleşmesini sağlar.
  - GoF örneği: çok satırlı bir metin alanına (TextArea/TextView) kimi zaman kaydırma çubuğu (yatay, dikey ya da ikisi), kimi zaman çerçeve, kimi zaman ikisi birden eklenmek isteniyor.
  - Çözüm 1, kalıtımla: `ScrollTextArea`, `BorderTextArea`, `ScrollBorderTextArea` kardeş alt sınıflar. Kaydırma ve çerçeve mantığı `ScrollBorderTextArea`'da tekrar yazılır (kopyala-yapıştır); yatay/dikey ayrımıyla sınıf sayısı daha da artar.
  - Çözüm 2, çoklu kalıtımla: `ScrollBorderTextArea` hem `ScrollTextArea`'dan hem `BorderTextArea`'dan kalıtılır. Zayıf yönleri (slayttan): çoklu kalıtım her dilde desteklenmez (Java ve C#'ta yok, C++'ta var); bu şekil "diamond problem" örneğidir; `Panel` gibi başka bir sınıfa da kaydırma ve çerçeve kazandırmak için üç sınıf daha gerekir ve bu durumda kaydırma ile çerçeve yeteneklerinin tek yerde gerçeklenmesi avantajı da kalkar.
  - Diamond problem: `TextArea`'da soyut olan bir metot iki ara sınıfta farklı gövdelerle tanımlanmışsa, en alttaki sınıfa hangisinin geçeceği belirsizdir. Diller bunu farklı çözer (ilk kalıtılan ebeveyn, son kalıtılan ebeveyn ya da C++'ta pure virtual ile alt sınıfa zorlamak); getirdiği karmaşıklık yararından fazla görüldüğü için güncel dillerde çoklu kalıtım desteklenmez.
- Decorator kalıbının yapısı (ekrandan, 00:20:00)



  - Fikir: sınıfları değil nesneleri birbirine bağlamak; her ek sorumluluk ayrı bir nesnede olur. Bir `TextView` bir `ScrollDecorator` nesnesiyle ilişkilendirilince kaydırma, ardından bir `BorderDecorator` ile ilişkilendirilince çerçeve yeteneği kazanır; başka bir `TextView` yalnız çerçeveyle süslenebilir.
  - Ortak üst sınıf (`VisualComponent`), hem süslenecek asıl nesneyi hem diğer dekoratörleri temsil eder. Kaydırma çubuğu gerektiren tek bileşen metin alanı olmayabilir (paneller, pencere çerçeveleri); dekoratörler başka görsel bileşenlerle de kullanılabilir.
  - Component soyut sınıf ya da arayüz olabilir. Decorator ise arayüz olamaz, soyut sınıf olmalıdır: ilişkili bileşeni tutan `component` alanını ve çekirdek işi bu bileşene ileten metot gövdesini içermesi gerekir. Slayttaki elmasa rağmen aradaki ilişki bir dizi değil, birebir sahipliktir.
  - Somut dekoratörler ek davranış, ek durum bilgisi (üye alan) ya da ikisini birden ekler.

Ek iş çekirdek işten önce, sonra ya da hem önce hem sonra yapılabilir. `BorderDecorator.Draw()` önce üst sınıfın `Draw()`'unu çağırır (çağrı zincir boyunca asıl bileşene kadar iner), sonra çerçeveyi çizer.

- Decorator ne zaman kullanılır, zayıf yönü
  - Tek tek nesnelere, aynı türdeki diğer nesneleri etkilemeden, şeffaf ve dinamik biçimde sorumluluk eklemek için; bazı sorumlulukları iptal etmek ya da askıya almak için (çekirdek işlemi çağırmamak); kalıtıma alternatif bir tasarım aranırken.
  - Gerçek kullanım: Java'nın akış (stream) tabanlı G/Ç kütüphanesi. Örneğin `FileOutputStream`, `ObjectOutputStream` ile zincirlenerek (süslenerek) nesneler diske yazılır.
  - Dikkat: çekirdek işlemin askıya alınması gerekmiyorsa somut dekoratörde çağrıyı unutup zinciri koparmamak gerekir; soyut Decorator'daki kod buna yardım eder ama alt sınıfta metot yeniden tanımlanırken unutulabilir.
- Decorator örneği: satış makbuzu, üstbilgi ve altbilgi (ekrandan, 00:30:00 ve 00:31:00)
  - Problem: `SalesOrder` satış emirlerini işler (`process`), makbuzu `SalesTicket.prtTicket()` basar. Sonradan makbuzlara üstbilgi (header) ve altbilgi (footer) eklenmek isteniyor.
  - Çözüm 1: `Header` ve `Footer` sınıfları; `prtTicket` önce `prtHeader`, sonra makbuz, sonra `prtFooter` çağırır. Her makbuza üstbilgi ya da altbilgi basılmayacaksa `hasHeader`, `hasFooter` parametreleri ve `if`'ler gerekir.
  - Çözüm 2: farklı üstbilgi ve altbilgi türleri için ayrı kalıtım hiyerarşileri (`Header` → `HeaderType1`, `HeaderType2`; `Footer` → `FooterType1`, `FooterType2`); `SalesTicket` `headerInfo` ve `footerInfo` tutar. Slayta göre bu henüz işlenmemiş Strategy kalıbının bir gerçeklemesidir. Hoca Abstract Factory, Factory Method ya da Builder'ın da düşünülebileceğini, bunların karşılaştırmasının ayrıca incelenebileceğini söyledi.
  - Asıl gereksinim: aynı makbuzda birden fazla üstbilgi ve/veya altbilgi basılması (ör. `HEADER1, HEADER2, Receipt, FOOTER2` ya da `HEADER1, Receipt, FOOTER1, FOOTER2`). Her birleşim için ayrı sınıf yazmak kopyala-yapıştır kod demektir; bu durumda gereksinimi ancak Decorator karşılar.
  - Önerilen çözüm:

```
Client ··> Component {+prtTicket()}  <──── 1 ────┐
             △── SalesTicket {+prtTicket()}       │
             △── TicketDecorator {-myComponent: Component, +prtTicket()} ◇ (slaytta 0..1;
                    △── Header1, Header2 {+prtTicket(), -prtHeader()}
                    △── Footer1, Footer2 {+prtTicket(), -prtFooter()}
TicketDecorator.prtTicket: myComponent null değilse myComponent.prtTicket() çağrılır
Header.prtTicket:  prtHeader(); TicketDecorator.prtTicket();
Footer.prtTicket:  TicketDecorator.prtTicket(); prtFooter();
```

  - Slayttaki çokluk `0..1` idi; hoca dekoratörün kurucusunda bileşeni parametre olarak aldığı için bileşensiz dekoratör oluşmadığını, çokluğun 1 olması gerektiğini söyledi. Çekirdek işi çağıran `callTrailer` metodundaki `null` denetimi, kurucuya `null` verilmesine karşı ek önlemdir.
  - Üstbilgi ve altbilgiyi basan metotlar, rastgele çağrılmasınlar diye private yapılabilir.
  - `Header1` kodu (ekrandan, 00:35:40); `Header2` aynıdır:

```java
package decorator.example;
public class Header1 extends TicketDecorator {
    public Header1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket () {
        // place printing header 1 code here
        System.out.println(getClass().getName());
        super.callTrailer();
    }
}
```

  - `getClass().getName()`, nesnenin sınıfının tam nitelikli adını (`decorator.example.Header1`) verir; örnekte hangi nesnenin ne zaman denetimi aldığını göstermek için kullanıldı. Footer'larda önce `super.callTrailer()`, sonra altbilgi yazma kodu gelir.

- Zinciri kuran `Factory` (ekrandan, 00:40:00):

```
package decorator.example;
public class Factory {
    public Component getComponent () {
        Component myComponent;
        myComponent= new SalesTicket();
        myComponent= new Footer1(myComponent);
        myComponent= new Header1(myComponent);
        return myComponent;
        //VEYA: Aşağıdaki kod aynı etkiyi yapar
        //return(new Header1(new Footer1(new SalesTicket())));
    }
}
```

  - Çalışma sonucu: `HEADER1`, `Receipt`, `FOOTER1`. Hangi makbuz türünün basılacağı çalışma anında kullanıcı tarafından belirlenir; örneğin bir açılır listenin her öğesine uygun biçimde zincirlenmiş bileşen bağlanabilir.

- Decorator'da zincirleme sırası
  - Bir header ile bir footer'ın veriliş sırası sonucu değiştirmez. İki header varsa sıra önemlidir: bileşene daha uzak (zincirin dışındaki) header önce basılır. `Header2`, sonra `Header1`, sonra `Footer1` ile süslenen makbuz `HEADER1, HEADER2, Receipt, FOOTER1` basar.
  - İç içe tek satır yazım (`new Header1(new Header2(new SalesTicket()))`), basım sırasını görselleştirmeyi kolaylaştırır: bileşene daha yakın olan header daha alta basılır.

- Facade kalıbı
  - Amaç: karmaşık bir alt sisteme tek bir erişim noktası (basit bir arayüz) vermek. Ör. 40 işlem 4 farklı sınıfa dağılmışsa, istemci hangi işin hangi sınıfta olduğunu bilmek zorunda kalmadan tek bir Facade sınıfı üzerinden çalışır.
  - Zorlamak istenirse alt sistem sınıfları bir pakete konur, Facade da o paketin üyesi yapılır ve diğer sınıfların görünürlüğü kısıtlanır.
  - Kullanım: karmaşık alt sisteme basit arayüz sarmak, alt sistemleri katmanlara ayırmak, istemcilerin sistemin yalnız bir kısmıyla konuşmasını sağlayıp bağlaşımı azaltmak. Farklı istemci türleri farklı alt sistemlerle konuşacaksa birden fazla Facade olabilir. Basit olduğu için örnek yapılmadı.

- Flyweight kalıbı: amaç ve örnek (ekrandan, 01:10:00)
  - Amaç: paylaşım yoluyla çok sayıda basit nesnenin kullanımını desteklemek. Paylaşılan nesnelerin her birine flyweight denir.
  - Örnek: font ve biçim bilgisine göre metni çok net gösteren (excellent rendering) bir metin editörü. Harfler grafik nesneleri olarak, metin de satır ve sütunlardaki harfler olarak ele alınır (anlatımı kolaylaştırmak için sabit genişlikli font varsayıldı). Her harf ayrı nesne olursa küçük bir belgede bile binlerce nesne olur; slayta göre üç sayfalık ders öneri formunda bile boşluklarla 5000 kadar harf vardır.
  - Aynı harfin farklı kopyaları yalnız satır ve sütun bilgisiyle ayrılır. "Değişeni bul ve sarmala": her harf için tek bir nesne (render edilmiş bitmap) bir havuzda tutulur; sayfadaki her konum havuzdaki nesneye yalnız bir işaretçi tutar.

- Flyweight'in yapısı (ekrandan, 01:20:00)

```
FlyweightFactory {GetFlyweight(key)}  ◇── flyweights ──>  Flyweight {Operation(extrinsicS
   GetFlyweight(key):                                        △── ConcreteFlyweight {Operat
     if (flyweight[key] exists) return existing flyweight;   △── UnsharedConcreteFlyweight
     else { create new flyweight; add it to pool; return it; }
Client ──> FlyweightFactory,  Client ──> ConcreteFlyweight,  Client ──> UnsharedConcreteF
```

  - Durum bilgisi ikiye ayrılır: içsel (intrinsic) durum nesnede saklanır ve aynı türden nesneler arasında değişmez (harfin bitmap'i); dışsal (extrinsic) durum dışarıdan, işlem metoduna parametre olarak verilir (satır ve sütun). Bu ayrım yapılamıyorsa Flyweight'in anlamı yoktur.

  - İçsel durum somut flyweight'ler arasında çok farklıysa tanımı alt sınıflara bırakılır; az farklıysa Flyweight soyut sınıfına bazı somut alanlar ve metotlar konabilir.
  - FlyweightFactory (registry) her flyweight'i tekil bir anahtarla tutar; `GetFlyweight(key)` varsa mevcut nesneyi döndürür, yoksa oluşturup havuza ekler. İsteğe bağlı olarak ekleme metodu da olabilir.
  - Somut flyweight'ler fabrikanın private iç sınıfı yapılıp public bir arayüz üzerinden kullanılırsa, nesnelere yalnız o arayüzle erişilebilir.
  - Her flyweight paylaşılmak zorunda değildir (`UnsharedConcreteFlyweight`); bir flyweight'e birden fazla istemci erişebilir.

- Flyweight ne zaman kullanılır, ödünleşim
  - Uygulama çok sayıda nesne kullanıyor ve saklama masrafı yüksekse; nesnelerin durum bilgisinin çoğu (en azından bir kısmı) dışsallaştırılabiliyorsa; dışsal durum ayrılınca çok sayıda nesne grubu yerine az sayıda paylaşılan nesne yetiyorsa; uygulama nesne kimliğine bağımlı değilse.
  - Zaman-bellek ödünleşimi: içsel bilgi azsa bellekten kazanılır ama çok bilgi çalışma anında hesaplanır; tasarruf az olacaksa kalıbı kurmaya gerek yoktur. İçsel durum üyeleri `final` tanımlanabilir.
  - Nesne kimliği örneği: bir web sitesinin toplam ziyaret sayısı ile tekil ziyaretçi sayısı. Tekil ziyaretçiyi saymak için her ziyaretçinin kimliği gerekir. Flyweight'te nesnenin kimliği yoktur, yalnız genel bir anahtarı (ör. `A` karakteri) vardır; kimlik gerekiyorsa kalıp kullanılamaz.

- Flyweight örneği: kimya fabrikası (ekrandan, 01:30:00, 01:36:30, 01:39:00, 01:41:15)
  - Problem: fabrika çok sayıda kimyasal madde ve bunlardan karışımlar üretir; aynı madde farklı karışımlarda farklı gramajla yer alır. Örnek Design Patterns Java Workbook kitabından alınmıştır.
  - İlk çözüm: `KimyasalMadde` (`gram`, `isim`, `simge`, `atomAgirligi`; `getMol()` = `gram/atomAgirligi`) ve `Karisim` (`isim`, `LinkedList` içinde maddeler, `maddeEkle(KimyasalMadde)`, `tarifEt()`). Örneğin 100 gr barutta 75 gr potasyum nitrat, 15 gr karbon, 10 gr kükürt bulunur. Sorun: karbon başka karışımlarda başka gramajla yer alınca yalnız gramı farklı, kalan bilgisi aynı çok sayıda karbon nesnesi oluşur.
  - İkinci adım (Extract Class yeniden düzenlemesi): değişen gram bilgisi yeni `Bilesen` sınıfına taşınır; `Bilesen` bir `KimyasalMadde`'ye birebir sahiptir.
  - Flyweight ile çözüm:

```
Karisim {-isim: String, -maddeler: LinkedList, +setIsim(String), +getIsim(): String,
         +maddeEkle(simge: String, gram: double), +tarifEt(): String}
   ◇──> Bilesen {-gram: double, -madde: KimyasalMadde, +Bilesen(double, KimyasalMadde),
                 +getGram(): double, +getMol(): double, +getMaddeAdi(): String}
          1 ──> 1 KimyasalMadde {-isim, -simge: String, -atomAgirligi: double,
                                 +getIsim(), +getAtomAgirligi(), +getMol(gram: double):
   ··> MaddeFabrikasi {-maddeler: HashMap<String, KimyasalMadde> {static},
                       +maddeBul(simge: String): KimyasalMadde {static}}
```

  - `maddeEkle` artık madde nesnesi değil, maddenin tekil tanımlayıcısı olan simgeyi alır ve `MaddeFabrikasi.maddeBul(simge)` ile paylaşılan nesneye ulaşır. Fabrika `C`, `S`, `KNO3`, `NaNO3` gibi maddeleri bir `HashMap`'te tutar; `maddeBul` yalnız `get` çağırır ve statik olabilir.

```java
public class Karisim {
   private String isim;
   private LinkedList<Bilesen> maddeler = new LinkedList<Bilesen>();
   public String getIsim() { return isim; }
   public void setIsim(String isim) { this.isim = isim; }
   public void maddeEkle( String simge, double gram ) {
      maddeler.add( new Bilesen(gram, MaddeFabrikasi.maddeBul(simge)) );
   }
   public String tarifEt( ) {
      String tarif = isim;
      for( Bilesen madde : maddeler )
         tarif += "\n" + madde.getGram() + "gr " + madde.getMaddeAdi();
```

```
        return tarif;
    }
}
```

  - `MainApp` "Kara barut" (`NaNO3` 75, `C` 15, `S` 10) ve "Temiz barut" (`KNO3` 75; derste çalıştırılan sürümde `C` 20, `S` 25) karışımlarını oluşturup `tarifEt()` çıktısını yazar (ekrandan, 01:41:15).
  - Kazanç: bir kimyasal madde ne kadar çok bileşende yer alıyorsa o kadar bellek tasarrufu sağlanır. Hoca `Bilesen`'li ara tasarım ile Flyweight'li tasarım arasındaki farkın küçük olduğunu, fabrikanın maddelere toplu ve tekil erişim noktası sunarak bir adım daha tasarruf sağladığını söyledi.

- Proxy kalıbı: amaç ve örnek (ekrandan, 02:00:00)
  - Amaç: bir nesneye erişimi denetlemek için onun yerine geçecek başka bir nesneyi araya koymak.
  - Örnek: kelime işlemcide büyük resimler belge açılırken hemen belleğe yüklenmez (yanıt süresini kısa tutmak, bellek tasarrufu). Belgede resmin yerine bir `ImageProxy` tutulur; sayfa çizileceği zaman proxy resmi dosyadan yükler ve çizdirir.

```
DocumentEditor ──> Graphic {Draw(), GetExtent(), Store(), Load()}
                     △── Image {Draw(), GetExtent(), Store(), Load(); imageImp, extent}
                     △── ImageProxy {Draw(), GetExtent(), Store(), Load(); fileName, exte
ImageProxy.Draw():      if (image == 0) { image = LoadImage(fileName); }  image->Draw()
ImageProxy.GetExtent(): if (image == 0) { return extent; } else { return image->GetExtent
```

  - Kalıp yapısı: gerçek varlığın (RealSubject) yetenekleri bir `Subject` arayüzünde listelenir; proxy'nin gerçek nesnenin yerine geçebilmesi için bu arayüz şarttır. Proxy bütün metotları gerçekler; işi gerçek nesneye iletir, öncesinde ve/veya sonrasında ek işler (yükleme, günlük tutma vb.) yapabilir. İstemci yalnız vekille ilişkilidir.
  - Diğer kullanımlar: nesneye basit bir işaretçiden fazlası gerektiğinde; nesne başka bir adres uzayında (dağıtılmış sistemde başka bir bilgisayarda) ise her bilgisayardaki vekil, ağ ayrıntılarını ve eşzamanlı değişiklik taleplerinin senkronizasyonunu istemciden soyutlar.

- Proxy örneği: izledikçe öde (pay-per-view) yayın sunucusu (ekrandan, 02:10:00)
  - Problem: yayın sunucusu istenen programı istekçinin adresine (akıllı televizyon, set üstü kutu, otel odası televizyonu) gönderir. Göndermeden önce abonelik ya da kredi ödemesinin denetlenmesi isteniyor; istekçi kodunun kaynağına erişim yok ya da çekirdek sunucunun kodu değiştirilmek istenmiyor (bazı müşterilere yalnız çekirdek ürün, bazılarına koruma ve ödeme yetenekli üst paket satılabilir). Bu tür proxy'ye protection proxy denir.

```
«interface» StreamSource {+stream(programID, destination: String): boolean}
   △┄ StreamServer {+stream(programID, destination: String): boolean}
   △┄ StreamProxy {-original: StreamSource, +StreamProxy(original: StreamSource),
                   +stream(programID, destination: String): boolean,
                   +checkPayment(programID, destination: String): boolean}  ──1──> Stream
StreamDestination {-server: StreamSource, -address: String, +StreamDestination(address: S
                   +getAddress(): String, +setServer(StreamSource), +makePayment(double):
                   +requestProgram(programID: String): boolean}  ──1──> StreamSource
MainApp ──> StreamDestination, StreamServer, StreamProxy
```

  - Proxy, orijinal sunucuyu somut sınıfı yerine arayüz türünden tutar; bağlaşımı artırmamak için somut sınıfa sahiplik gerekmez. `checkPayment`, hedefin yeterli kredisi varsa `true` döndürür. `StreamDestination` da sunucuyu arayüz üzerinden tutar, çünkü ona sunucunun kendisi değil proxy verilir. Kod iskelet olarak yazıldı; `MainApp` bir sunucu, bir proxy ve bir istemci oluşturup istemciye sunucu olarak proxy'yi verir.

- Davranışsal (behavioral) kalıplara giriş
  - Ortak nokta: nesnelerin davranışlarını çalışma anında değiştirebilmelerini sağlamak. Decorator da bunu yapar ama ana katkısı bakımından yapısal sayılır. Hoca kalıpları

üç gruptan birine kesin yerleştirmeyi sevmediğini, kalıbın hangi gruba girdiğini ezberlemenin istenmediğini söyledi.

- Chain of Responsibility kalıbı (ekrandan, 02:24:00)
  - Amaç: birbiriyle ilişkili bir nesne zincirinde (ya da hiyerarşisinde), bir isteği gereğince yerine getiremeyen nesnenin isteği, gerekli yetki ya da bilgiye sahip olan bir sonrakine iletmesi. Metot bütün nesnelerde tanımlıdır; mesele metodun var olmaması değil, yetki ya da bilginin yetersizliğidir.
  - Örnek: bağlama duyarlı yardım. Soru işaretine tıklayıp bir iletişim kutusundaki bir denetime tıklayınca o denetimin yardımı gösterilir; denetime özel yardım yoksa istek içinde bulunduğu iletişim kutusuna, o da yoksa uygulamanın ana yardımına iletilir (`HandleHelp`). Bileşenler büyük olasılıkla zaten bir Composite yapısının parçalarıdır; ortak arayüz yoksa `if/else`, `switch`, tür denetimi ve dönüşümü gibi kötü kod gerekir.

```
Client ──> Handler {HandleRequest()} ── successor ──> Handler
              △── ConcreteHandler1 {HandleRequest()}
              △── ConcreteHandler2 {HandleRequest()}
aClient.aHandler ──> aConcreteHandler.successor ──> aConcreteHandler.successor
```

  - Bileşenler (slayttan): Client yanıtlanacak mesajın kaynağıdır; Handler yanıtlanacak mesajı tanımlayan arayüzdür, zincir bağlantısını (successor) da soyut sınıf olarak gerçekleyebilir; ConcreteHandlerX mesajı yanıtlar ya da bağlantılı olduğu bir başkasına iletir.
  - Kullanım: bir mesajı karşılayabilecek birden fazla nesne varsa ve en iyi yanıtlayacak nesne önceden bilinemiyorsa.
  - Zayıf yön: zincir doğru yapılandırılmazsa isteğin yanıtlanacağının garantisi yoktur; doğru bir sonlandırma gerekir.
  - Zincir hazır değilse, zinciri kurma ve ilerletme mekanizması en uygun yer olan soyut Handler'a kodlanır ve kalıtımla alt sınıflara aktarılır. Birden fazla istek türü varsa `HandleRequestX` gibi metotlar aynı mantıkla çalışır.

- Chain of Responsibility örneği: reklam yönetim yazılımı (ekrandan, 02:30:00–02:50:00)
  - Problem (hocanın kendi örneği): farklı web sitelerinde kampanya kurallarına göre reklam gösterilecek. Kampanya 1: spor sitesinin 18-39 yaş arası erkek ziyaretçilerine tıraş bıçağı, 18-39 yaş arası kadın ziyaretçilerine fitness salonu, diğerlerine spor mağazasındaki indirimin reklamı. Kampanya 2: haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamı, aksi hâlde Kampanya 1'i aynen uygula. Kampanya 3: ziyaretçinin coğrafi bölgesi ve gelir düzeyine göre portföydeki lokanta reklamlarından biri.
  - Hoca bunun Decorator olmadığını vurguladı: Decorator'da zincirdeki her bileşen kendi işini yapar, Chain of Responsibility'de ise yalnız biri yanıt verir.
  - Tasarım: `User` (cinsiyet, gelir düzeyi, yaş; `toString()`, `yasDahilMi(alt, ust)`), `Cinsiyet` (KADIN, ERKEK) ve `GelirDuzeyi` (A, B, C) enum'ları, soyut `Reklam` (`banner`, `sonraki`), alt sınıfları `TrasBicak`, `FitnessSalonu`, `IndirimSporMagaza`, `OtoDergi`.

```
public Reklam(Banner banner, Reklam sonraki) {
    this.banner = banner; this.sonraki = sonraki;
}
public abstract boolean reklamUygunMu ( User u );
public abstract void reklamGoster( User u );
public final void reklamKotar( User u ) {
    if( reklamUygunMu(u) ) {
        System.out.println( this.getClass().getName()
            + "->" + u );
        reklamGoster(u);
    }
    else if( sonraki != null ) {
        System.out.println( this.getClass().getName()
            + "->" + sonraki.getClass().getName());
        sonraki.reklamKotar(u);
    }
```

```
}
```

(kod: ekrandan, 02:36:30)

- Zinciri kuran ve ilerleten `reklamKotar`, alt sınıflar müdahale edemesin diye `final`'dır. `FitnessSalonu.reklamUygunMu`: cinsiyet `KADIN` ve `yasDahilMi(18, 39)` ise `true` (ekrandan, 02:41:00). `IndirimSporMagaza` her kullanıcı için `true` döndürür; bu, zincirin anlamlı biçimde sonlanmasını sağlar.
- Zincir sırası önemlidir: herkese uygun olan indirim reklamı zincirin başına konursa diğer reklamlar hiç gösterilmez.
- `kampanya1()` (ekrandan, 02:50:00): `reklamlar[2] = IndirimSporMagaza(..., null), reklamlar[1] = FitnessSalonu(..., reklamlar[2]), reklamlar[0] = TrasBicak(..., reklamlar[1]);` üç kullanıcı (Erkek-C-23, Erkek-A-53, Kadın-B-33) `reklamlar[0].reklamKotar(...)` ile zincire verilir. `kampanya2()` başa `OtoDergi` ekler.
- Hoca reklamları dizi yerine bağlı listede tutup Kampanya 1'in listesini Kampanya 2'de yeniden kullanmanın daha iyi olacağını, dizinin kopyaya yakın (duplicate) kod ürettiğini söyledi.

Hocanın Özellikle Vurguladığı Kısımlar

- Decorator: Decorator sınıfı arayüz olamaz
  - İlişkili bileşeni tutan alan ve çekirdek işi bu bileşene ileten kod soyut Decorator sınıfında bulunmalıdır. Component ise arayüz olabilir.
- Decorator'da ek işin yeri ve zincir sırası
  - Header'larda ek iş çekirdek işten önce, footer'larda sonra yapılır. Aynı türden iki dekoratörde (iki header) süsleme sırası basım sırasını belirler; header ile footer arasında sıra önemli değildir.
  - Askıya alma gerekmiyorsa çekirdek işlemin çağrılması unutulmamalı, zincir koparılmamalıdır.
- Decorator ile Chain of Responsibility farkı
  - Decorator'da zincirdeki bütün nesneler işini yapar; Chain of Responsibility'de yalnız biri yanıt verir. Reklam problemi Decorator'a uymaz.
- Çoklu kalıtım ve diamond problem
  - Getirdiği belirsizlik ve karmaşıklık yüzünden güncel dillerde desteklenmez; Decorator gibi daha iyi çözümler vardır.
- Flyweight'in ön koşulu: içsel/dışsal durum ayrımı
  - Bu ayrım yapılamıyorsa kalıbın anlamı yoktur. Ayrıca uygulama nesne kimliğine bağımlı olmamalıdır.
  - Zaman-bellek ödünleşimi gözetilmeli; tasarruf azsa kalıp kurulmaz.
- "Neyin değiştiğini bul ve onu sarmala"
  - Flyweight'te harfin konumu, kimya örneğinde gram bilgisi ayrı modellendi.
- Chain of Responsibility'de zincirin doğru kurulması
  - Doğru sonlandırma olmazsa istek yanıtsız kalabilir; herkese uygun bir halka zincirin başına konursa diğerleri hiç çalışmaz. Zincir mekanizması üst sınıfta ve `final` olmalıdır.
- Kalıtım yerine parça-bütün (sahiplik) ilişkisi
  - Hoca mümkün olduğunca kalıtım yerine sahiplik ilişkilerini kullanmaya çalıştıklarını yineledi.
- Kalıpları gruplara göre ezberlemek gerekmez
  - Bir kalıbın yaratımsal, yapısal ya da davranışsal gruptan hangisine girdiği sorulmaz.

Kısa Tekrar Notları

- Decorator: nesnelere çalışma anında, tek tek, sorumluluk ekler. Component (ortak arayüz), ConcreteComponent (asıl nesne), Decorator (soyut, `component` alanı, işi iletir), ConcreteDecorator (ek davranış ve/veya ek durum).
- Ek iş çekirdek işten önce, sonra ya da iki tarafta yapılabilir; çekirdek iş çağrılmazsa sorumluluk askıya alınmış olur.
- Java G/Ç akışları Decorator kullanır (`FileOutputStream` + `ObjectOutputStream`).

- Makbuz örneği: `new Header1(new Footer1(new SalesTicket()))` → `HEADER1, Receipt, FOOTER1`.
- Facade: karmaşık alt sisteme tek erişim noktası; bağlaşımı azaltır, alt sistemleri katmanlar.
- Flyweight: paylaşılan çok sayıda basit nesne; içsel durum nesnede, dışsal durum parametreyle; FlyweightFactory anahtarla havuzdan verir ya da oluşturup ekler.
- Flyweight koşulları: çok nesne, yüksek saklama masrafı, dışsallaştırılabilir durum, nesne kimliğine bağımlı olmamak.
- Kimya örneği: gram → `Bilesen` (Extract Class); `MaddeFabrikasi.maddeBul(simge)` paylaşılan `KimyasalMadde`'yi döndürür.
- Proxy: gerçek nesnenin yerine geçen vekil; aynı `Subject` arayüzünü gerçekler, işi iletir, önüne/arkasına ek iş koyar. Örnekler: geç yüklenen resim (`ImageProxy`), dağıtılmış nesne, protection proxy (`StreamProxy.checkPayment`).
- Davranışsal kalıplar: nesnelerin davranışını çalışma anında değiştirebilmesi.
- Chain of Responsibility: Handler + successor; isteği karşılayamayan bir sonrakine iletir. Zincir doğru kurulmalı ve sonlanmalı.
- Reklam örneği: `reklamKotar` final; `IndirimSporMagaza` herkese uygun, zincirin sonunda.

Detaylı Açıklamalar

- Decorator kalıbının çıkış noktası, kalıtımın sınıf düzeyinde ve derleme anında belirlenen bir ilişki olmasıdır. Bir metin alanına kaydırma çubuğu ve çerçeve gibi yetenekleri farklı birleşimlerle eklemek kalıtımla yapılırsa her birleşim için ayrı bir sınıf gerekir ve ortak mantık sınıflar arasında kopyalanır. Çoklu kalıtım kopyalamayı önler ama diamond problem belirsizliğini getirir ve çoğu güncel dilde yoktur. Decorator ise sınıfları değil nesneleri bağlar: her ek yetenek ayrı bir dekoratör nesnesidir ve bu nesneler süsledikleri bileşeni bir alanda tutar. Dekoratör de bileşenle aynı üst türden olduğu için başka bir dekoratörün bileşeni olabilir; böylece istenen sırada ve birleşimde bir nesne zinciri kurulur. Zincirin ucundaki nesneye yapılan çağrı her dekoratörün ek işini yaparak asıl bileşene kadar iner.
- Makbuz örneğinde bu yapı somut olarak görüldü. `TicketDecorator` soyut sınıfı `myComponent` alanını tutar ve kurucusunda bileşeni alır; `callTrailer` metodu bileşen `null` değilse onun `prtTicket` metodunu çağırır. `Header1` ve `Header2` önce kendi başlıklarını yazar, sonra `callTrailer` çağırır; `Footer1` ve `Footer2` önce `callTrailer` çağırır, sonra altbilgiyi yazar. Bu yüzden zincirde header ile footer'ın sırası sonucu değiştirmez, ama iki header'dan dışta olan önce basılır. Hoca hangi birleşimin kullanılacağının çalışma anında, örneğin bir açılır listedeki öğelere uygun zincirler bağlanarak seçilebileceğini, bunun için karmaşık `switch-case` yapılarına gerek olmadığını belirtti.
- Facade, alt sistemin sınıflarını değiştirmeden önlerine tek bir sınıf koyar. İstemci, hangi işlemin hangi sınıfta olduğunu öğrenmek yerine Facade'ın metotlarını çağırır; Facade da isteği uygun alt sistem sınıfına yönlendirir. Böylece istemci ile alt sistem arasındaki bağlaşım azalır.
- Flyweight'in temeli, nesnenin durum bilgisini paylaşılabilir ve paylaşılamaz iki parçaya bölmektir. Metin editörü örneğinde harfin render edilmiş görüntüsü bütün `A`'lar için aynıdır (içsel durum), konumu ise her `A` için farklıdır (dışsal durum). İçsel durum tek bir paylaşılan nesnede tutulur, dışsal durum çizim anında parametre olarak verilir. Paylaşılan nesnelere tekil bir anahtarla erişmek için bir fabrika (registry) gerekir: istenen anahtar havuzda varsa o nesne döner, yoksa oluşturulup havuza eklenir. Kalıp, bellek kazancı ile çalışma anında hesaplama maliyeti arasında bir ödünleşimdir; nesne kimliğinin önemli olduğu durumlarda (ör. tekil ziyaretçi sayımı) kullanılamaz.
- Kimya fabrikası örneği aynı fikri adım adım gösterdi. İlk tasarımda gram bilgisi `KimyasalMadde` içinde olduğu için aynı madde her karışımda ayrı bir nesne olarak kopyalanıyordu. Değişen bilgi (gram) `Bilesen` sınıfına çıkarıldı (Extract Class); `Bilesen` gramı ve paylaşılan `KimyasalMadde` nesnesini tutar. Son adımda `Karisim.maddeEkle` madde nesnesi yerine simge almaya başladı ve maddeyi `MaddeFabrikasi`'nın statik `HashMap`'inden `maddeBul` ile getirdi. Böylece her kimyasal madde bellekte bir kez bulunur; madde ne kadar çok karışımda geçerse kazanç o kadar büyür.
- Proxy kalıbında vekil, gerçek nesneyle aynı arayüzü gerçeklediği için istemci farkı anla-

maz. Resim örneğinde `ImageProxy` resmi ancak çizileceği zaman yükler; `GetExtent` resim yüklenmemişse proxy'de saklanan `extent` değerini, yüklenmişse gerçek resmin değerini döndürür. Pay-per-view örneğinde `StreamProxy`, `stream` çağrısını orijinal sunucuya iletmeden önce `checkPayment` ile ödemeyi denetler. İstemci (`StreamDestination`) sunucuyu `StreamSource` arayüzü üzerinden tuttuğu için ona sunucu yerine proxy verilebilir ve istemcinin kodu değişmez. Hoca proxy'nin orijinal nesneyi somut sınıf yerine arayüz türünden tutmasının bağlaşımı azalttığını da belirtti.

- Chain of Responsibility, isteği gönderenle isteği karşılayacak nesne arasındaki bağı gevşetir. Her halka aynı metoda sahiptir; isteği karşılayabiliyorsa karşılar, karşılayamıyorsa bir sonrakine iletir. Reklam örneğinde bu mekanizma soyut `Reklam` sınıfındaki `final reklamKotar` metodunda toplandı: reklam kullanıcıya uygunsa gösterilir, değilse istek `sonraki` reklama iletilir. Somut reklamlar yalnız `reklamUygunMu` ve `reklamGoster` metotlarını gerçekler. Zincirin sonuna herkese uygun bir reklam konarak her isteğin yanıtlanması garanti edildi. Çalıştırmada Erkek-C-23 kullanıcısına tıraş bıçağı, Erkek-A-53 kullanıcısına (tıraş bıçağı ve fitness salonundan geçerek) spor mağazası indirimi, Kadın-B-33 kullanıcısına fitness salonu reklamı gösterildi. Kampanya 2'de zincirin başına eklenen otomobil dergisi reklamı, bu kullanıcıların hiçbirine uymadığı için gösterilmedi; ilk kullanıcının gelir düzeyi A yapılınca otomobil dergisi reklamı ona gösterildi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.